Package 'micsPlusTableR'

September 27, 2026

Title MICS Plus Excel Tabulation and Review Application

Version 0.4.0

Description Provides the MICS Plus Tabulation application for preparing, calculating, checking, exploring, and exporting survey tables from an Excel tabulation plan, survey data, and a survey-specific preparation script. The package is built primarily to support this browser-based application. Its documented R functions expose the supporting engine for reference, testing, and workflows run directly in R.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

Depends R (>= 4.2.0)

Imports bslib, dplyr, flextable, haven, here, htmltools, htmlwidgets, janitor, jsonlite, later, officer, openxlsx2, purrr, reactable, readr, readxl, rlang, shiny, shinyFiles, stringr, tibble, tidyr, tidyxl, ggplot2, Hmisc, labelled, memisc, reshape2, RM.weights, survey, srvyr

Suggests testthat (>= 3.0.0)

Config/micsPlusTableR/prep-packages ggplot2, Hmisc, labelled, memisc, reshape2, RM.weights, survey, srvyr

Config/testthat/edition 3

NeedsCompilation no

Config/build/clean-inst-doc FALSE

Config/roxygen2/version 8.1.0

Contents

micsPlusTableR-package 2
as_sdmx_compatible 5
calc_cells 5
check_mics_table 6
check_prep_dependencies 7
col_condition_f 8
col_group_perc_total_check 8
col_group_total_check 9
compare_mics_tables 10

create_excel_workbooks . . . 10
download_survey_files . . . 11
extract_var . . . 13
filter_second . . . 13
get_blank_cols . . . 14
install_prep_dependencies . . . 14
list_survey_files . . . 15
mics_session . . . 16
mics_table_name . . . 17
normalize_condition_text . . . 17
offline-workflow . . . 18
package-dependencies . . . 20
pivot_mics_table . . . 21
prep-script-dependencies . . . 21
prepare_mics_data . . . 23
read_mics_tabulation . . . 23
read_previous_mics_table . . . 24
row_condition_f . . . 25
row_group_perc_total_check . . . 26
row_group_total_check . . . 26
row_indent_group_perc_total_check . . . 27
row_indent_group_total_check . . . 28
run_app . . . 28
statistic-precision . . . 29
statistic-types . . . 30
survey_choices . . . 32
tabulate_extra_table . . . 33
tabulate_h . . . 33
tabulate_mics_table . . . 34
tabulate_v . . . 35
validate_sdmx_compatible . . . 35
worksheet-conditions . . . 36
write_mics_footnotes . . . 38
write_mics_table . . . 38

Index **40**

`micsPlusTableR-package`

micsPlusTableR: MICS Plus Excel tabulation and review

Description

micsPlusTableR is the R package behind the MICS Plus Tabulation application. Its main purpose is to provide a browser-based interface for preparing survey data, calculating tables from a tabulation plan, checking results, exploring data, and saving outputs when needed. Start the application with `run_app()`. R performs the calculations; Shiny provides the interface running in the browser.

The app and the supporting functions

The package was created primarily to serve the application. Most other functions perform background tasks used by, or exposed from, its tabulation engine. Their help pages document how the system works and support testing, diagnosis, and workflows run entirely in R. App users do not need to call these functions individually or follow the function index as a workflow.

Use the bundled User's Guide for setup, tab-by-tab instructions, and understanding a tabulation plan. Use this reference manual for exact function arguments, return values, examples, and engine behavior.

Running the application

Launch the Shiny application with `run_app()`:

```
micsPlusTableR::run_app()
```

Scripting workflow

For work directly in R, use an isolated session and call the required steps. A session is an ordinary R environment holding your data, plan, and results. `mics_session()` does not start Shiny or open a browser. An empty session is enough to read and inspect a worksheet; calculation additionally requires prepared data. Use `mics_session(hh = hh, hl = hl)` for data already prepared in your workspace, or `prepare_mics_data()` for raw survey files. These interfaces also make the app's engine available for testing:

- `mics_session()` creates an isolated session.
- `prepare_mics_data()` runs the preparation script.
- `read_mics_tabulation()` reads a worksheet's instructions.
- `tabulate_mics_table()` calculates cell results.
- `check_mics_table()` checks consistency.
- `pivot_mics_table()` arranges results for review.
- `create_excel_workbooks()` creates output workbooks.
- `write_mics_table()` writes the results.
- `write_mics_footnotes()` writes table footnotes.
- `read_previous_mics_table()` reads an earlier output.
- `compare_mics_tables()` compares two outputs.

See offline-workflow for a complete scripting example.

Individual steps and testing

`tabulate_v()` and `tabulate_h()` calculate a loaded plan in a specific direction; `calc_cells()` tests cells against explicit input data. `tabulate_extra_table()` maps supplied values to the loaded plan. `row_condition_f()` parses row predicates. `col_condition_f()` parses column predicates.

`normalize_condition_text()` normalizes Excel text. `extract_var()` identifies variables.

`get_blank_cols()` supports worksheet cleanup. `filter_second()` supports comparison cleanup.

Run a single consistency check:

- `row_group_total_check()` checks row counts.
- `row_group_perc_total_check()` checks row percentages.

- col_group_total_check() checks column counts.
- col_group_perc_total_check() checks column percentages.
- row_indent_group_total_check() checks indented counts.
- row_indent_group_perc_total_check() checks indented percentages.

Each returns status, issue count, all comparisons, and failing comparisons. Stateful helpers take a session. See offline-workflow for individual-step examples. The calc_cells() help page includes a runnable weighted example.

Dependencies

package-dependencies explains required app/engine packages and development tools. prep-script-dependencies documents survey packages separately. Both dependency groups install with the package; no separate preparation setup is needed.

For diagnosis, use check_prep_dependencies(). For repairs or extra custom-script requirements, use install_prep_dependencies().

Survey and wave configuration

Maintain inst/extdata/survey_choices.csv in the package source. It remains a plain, editable CSV on GitHub and is bundled automatically on installation. See survey_choices for its columns, validation rules, and update workflow.

Outputs and interpretation

Outputs include Excel workbooks and long-format observations. The app offers Excel, CSV, and RDS downloads of combined calculated table cells. These are aggregate table results, distinct from the survey records shown in Data View. Use as_sdmx_compatible() to convert observations to the project's SDMX-compatible contract. Use validate_sdmx_compatible() to check them. This contract does not by itself establish compliance with an external SDMX data structure. Table checks assess consistency; analysts remain responsible for preparation scripts, survey design, denominators, and interpretation of estimates.

Author(s)

Maintainer: MICS Plus Tabulation Team <micsplustabler@example.org>

Authors:

- MICS Plus Tabulation Team <micsplustabler@example.org>

See Also

run_app(), statistic-types, statistic-precision, survey_choices, offline-workflow

Examples

```
session <- mics_session()
session
system.file("extdata", "survey_choices.csv", package = "micsPlusTableR")
```

as_sdmx_compatible *Add an SDMX-compatible observation profile to MICS cell results*

Description

Converts either the package's engine cell results or an already standardized MICS cell table into a 35-variable long-format contract adapted from the mics-translation project. The output omits the custom `table_id`, `value`, `estimate`, `digits`, and `unw` duplicates; use `TABLE_ID`, `OBS_VALUE`, `DECIMALS`, and `denominator_n` instead. Input fields are unchanged. Source coordinates remain in `row_order` and `column_order`; the redundant `row_id`, `column_id`, `ROW_ID`, and `COLUMN_ID` fields are omitted. The output also omits `unweighted_n`, which duplicates `denominator_n` for engine cells. For standardized input, the supplied `denominator_n` is retained. The optional input `unit_measure` override is applied to `UNIT_MEASURE` but is not included as a separate output field.

Usage

```
as_sdmx_compatible(cells, config, table_id = NULL, table_context = NULL)
```

Arguments

`cells`	A data frame with one row per table cell.
`config`	A list containing `frequency`, `ref_area`, `survey_id`, and `time_period`.
`table_id`	Table identifier. Required for raw engine cell results.
`table_context`	Optional parsed tabulation context used to recover multi-level column headers and suppression settings.

Value

A tibble with one row per observation and the 35 standardized fields.

calc_cells *Calculate individual horizontal table cells*

Description

Calculate individual horizontal table cells

Usage

```
calc_cells(df, tab_r, tab_c3, tab, weight_var, weighted = FALSE)
```

Arguments

`df`	Prepared data frame, after block filters and calculations.
`tab_r`	Row specification with `row_index` and `row_lgc`.
`tab_c3`	Column specification with `col_index`, `col_condition`, and `col_var_name`; optional `col_logic` preserves display labels.
`tab`	Cell specification with `row_index`, `col_index`, and `stat_type`.

weight_var	Weight-column name. Unweighted calculations may use NA. The p_sum statistic always needs a weight column.
weighted	FALSE for unweighted results, TRUE for weighted results, or "both" (legacy mode, returns the unweighted result only).

Details

This background helper is available for explicit cell tests and code-only work; app users normally use Tabulator. Supported statistics include counts, indicator percentages, numeric means, medians, and constant totals. See statistic-types for the complete horizontal forms and their populations. Row mutations in row_lgc execute in Excel row order before row predicates and remain available to subsequent rows. With n_unw or n_unw1, the column condition hhmembers instead sums HLnum for row-selected records with total == 1. This is a household member base from household records, not an ordinary record count. Worksheet aliases such as total1 == 1 and totalHL == 1 are converted by read_mics_tabulation(), not this helper; see worksheet-conditions.

Bare mean is an indicator percentage; mean(variable) averages the named variable. Median and mean_unw forms are unweighted. The p_sum statistic divides the variable sum by the weight sum within both conditions, then multiplies by 100.

Missing statistic cells return NA. An empty denominator can produce NaN. Optional d= arguments are returned as display_digits metadata and never round the calculated value; see statistic-precision.

Value

A tibble with row/column indices, logic, statistic, and numeric value.

Examples

```
calc_cells(data.frame(total = c(1, 1)),
  data.frame(row_index = 9L, row_lgc = "TRUE"),
  data.frame(col_index = 3L, col_condition = "total == 1", col_var_name = "total_1"),
  data.frame(row_index = 9L, col_index = 3L, stat_type = "n_unw"),
  weight_var = NA_character_)

# Weighted count of sex == 2: 5. Use weighted = FALSE for a count of 2.
calc_cells(
  df = data.frame(total = 1, sex = c(1, 2, 2), weight = c(1, 2, 3)),
  tab_r = data.frame(row_index = 9L, row_lgc = "TRUE"),
  tab_c3 = data.frame(col_index = 3L, col_condition = "sex == 2",
                      col_var_name = "sex_2"),
  tab = data.frame(row_index = 9L, col_index = 3L, stat_type = "n"),
  weight_var = "weight", weighted = TRUE
)
```

---

check_mics_table *Run all applicable consistency checks*

Description

Runs the six group-count and percentage checks used by the application. Direct R calls retain engine diagnostics; the app suppresses console output while showing check results in its interface. Non-applicable checks are not passing validations. Details retain internal column names such as diff_value; the app displays readable labels such as Difference.

Usage

```
check_mics_table(session, table = NULL, tolerance = 1e-06)
```

Arguments

session	A tabulation session.
table	Cell results; defaults to the session's latest result.
tolerance	Maximum absolute difference treated as consistent.

Value

A list containing a summary data frame and detailed check tables.

---

check_prep_dependencies

Check dependencies of a preparation script and its helpers

Description

Inspect preparation code without executing it, then check whether each required package can be loaded. App dependencies and survey-specific preparation dependencies install together and are documented separately in prep-script-dependencies. This is a diagnostic tool; preparation checks dependencies automatically.

Usage

```
check_prep_dependencies(prep_script, stop_on_missing = FALSE)
```

Arguments

prep_script	Path to a preparation R script.
stop_on_missing	Stop with an installation command if a required package is missing or cannot be loaded. Defaults to FALSE for inspection.

Details

Reads the script's literal `prep_dependencies` declaration, qualified calls, library/require calls with literal package names, and known legacy survey/FIES function calls. It also inspects all R files in the sibling `FIES-inputs` folder and recursively follows literal source paths. Dynamically computed package names and source paths require an explicit `prep_dependencies` declaration in the main script. This checks package availability, not numerical correctness or version compatibility of the survey code. It never executes preparation code.

Value

A data frame with `package`, `required_by` (script basenames), `available`, and `problem` (NA when loadable).

See Also

`install_prep_dependencies()`, `prepare_mics_data()`

col_condition_f *Parse column conditions from a plan*

Description

Parse column conditions from a plan

Usage

```
col_condition_f(tab_c)
```

Arguments

tab_c	Data frame containing col_index and col_lgc.

Details

Separates source, filter, calculation, and weight instructions from the complete column predicate. Use - - - between setup instructions and the predicate. Multiline mutate(...) calls and predicates are supported; predicates such as x == 1 & followed by y == 2 retain both clauses. Parses text without evaluating it against survey data. Worksheet total aliases are converted by read_mics_tabulation() before this parser is called; this helper alone does not convert them.

Value

A tibble containing the column index, original condition, cleaned condition, and generated variable name.

See Also

row_condition_f(), normalize_condition_text(), worksheet-conditions

col_group_perc_total_check *Check that column-block percentages sum to 100*

Description

Check that column-block percentages sum to 100

Usage

```
col_group_perc_total_check(session, cell_results = NULL, diff = 1e-11)
```

Arguments

session	A tabulation session. Indent checks also require the loaded plan's indent_rows metadata.
cell_results	Long-format numeric results, defaulting to the latest result in the session. Use the complete results with logic/group metadata.
diff	Maximum absolute difference accepted as consistent.

Details

Runs only this check and updates its corresponding session check state. A missing total or distribution produces not_applicable. Use `check_mics_table()` to run all six checks together.

Value

A list with `status` (pass, fail, or not_applicable), `issue_count`, `details` (all comparisons), and `issues` (failing comparisons).

`col_group_total_check` *Compare column-group counts with the overall total*

Description

Compare column-group counts with the overall total

Usage

```
col_group_total_check(session, cell_results = NULL, diff = 1e-11)
```

Arguments

`session`	A tabulation session. Indent checks also require the loaded plan's `indent_rows` metadata.
`cell_results`	Long-format numeric results, defaulting to the latest result in the session. Use the complete results with logic/group metadata.
`diff`	Maximum absolute difference accepted as consistent.

Details

Runs only this check and updates its corresponding session check state. A missing total or distribution produces not_applicable. Use `check_mics_table()` to run all six checks together.

Value

A list with `status` (pass, fail, or not_applicable), `issue_count`, `details` (all comparisons), and `issues` (failing comparisons).

compare_mics_tables *Compare two generated tables*

Description

Compare two generated tables

Usage

```
compare_mics_tables(
  previous,
  current,
  ignore_col_names = FALSE,
  show_diff_over = 0
)
```

Arguments

previous	Previous or approved table.
current	Current table.
ignore_col_names	Compare columns by position instead of name.
show_diff_over	Minimum absolute numeric difference to report.

Value

A data frame containing differing cells.

create_excel_workbooks *Create output workbooks from a tabulation plan*

Description

Copies the tabulation plan to standard output names without overwriting an existing file. A numeric suffix is added after a filename collision.

Usage

```
create_excel_workbooks(
  path_tab_excel,
  output_dir,
  country_code,
  period,
  wave,
  target = c("both", "output", "formatted"),
  time_tag = format(Sys.time(), "%Y%m%d%H%M%S")
)
```

Arguments

path_tab_excel	Existing .xls or .xlsx tabulation plan.
output_dir	Directory where workbooks will be created.
country_code	Short country code used in filenames.
period	Survey period used in filenames.
wave	Wave label used in filenames.
target	One of "output", "formatted", or "both".
time_tag	Timestamp used in filenames.

Details

Creates copies of the tabulation plan; it does not calculate or populate table results. In the app, Write to Excel calls it on request, and Multi-Sheet Tabulator calls it automatically for unset required destinations. In R workflows, call it before write_mics_table unless using an existing workbook. Unrequested entries in the returned list are NULL.

Value

A list with output and formatted paths.

---

download_survey_files *Download survey tabulation plans and preparation files*

Description

Save selected plans, preparation scripts, and supporting reference assets from the public survey files repository. Household and household-member microdata must be obtained separately with the required permission.

Usage

```
download_survey_files(
  surveys = NULL,
  files = NULL,
  dest_dir = "inputs",
  overwrite = FALSE,
  quiet = FALSE,
  repo = "bariscr/micsPlusTableR-Files",
  ref = "main"
)
```

Arguments

surveys	Character vector of root survey folder names, such as "Jamaica (2023-24) Wave 1". NULL selects all surveys. Legacy codes for the initial published surveys (for example, "JAM_W1") remain accepted.
files	Character vector of exact repository-relative paths from list_survey_files(). NULL selects all files in the selected surveys.

dest_dir	Destination folder, relative to the current R working directory unless an absolute path is supplied. Defaults to "inputs".
overwrite	Replace existing files? Defaults to FALSE; any existing selected file stops the operation before downloading file contents.
quiet	Suppress file download progress and the completion message?
repo	GitHub repository in owner/name form.
ref	Branch, tag, or commit to download. Defaults to "main".

Details

Survey folders use the country, survey period, and wave, such as Jamaica (2023-24) Wave 1. Legacy code-and-wave folders are also supported. Downloads include root-level tabulation plans (.xls, .xlsx) and .R scripts or .RData reference assets inside prep-files-* subfolders. Other files, including SPSS microdata and hidden files, are excluded. Preparation subfolders are preserved. A single Git commit is resolved before listing or downloading, so changes to the branch during a download cannot mix remote versions. Newly published survey folders are discovered without a package update.

Downloads are staged and their sizes checked before destination files are written. A network failure leaves existing destination files unchanged. Files are downloaded in binary mode and are never executed. Selecting individual files may omit preparation helpers; select a whole survey folder for all its published materials. Microdata must still be obtained separately. Git LFS pointers are not supported.

Internet access is required. Unauthenticated GitHub API rate limits apply; if a request fails, check the connection and public repository/ref, or retry later. The browser ZIP link in the user's guide is an alternative.

Value

Invisibly returns the selected file-list rows with survey, path, size, url, and local_path columns, plus the resolved commit attribute.

See Also

list_survey_files()

Examples

```
## Not run:
download_survey_files("Jamaica (2023-24) Wave 1")
download_survey_files(c("Jamaica (2023-24) Wave 1",
                        "Mongolia (2025-26) Wave 2"), dest_dir = "survey-inputs")
download_survey_files() # All surveys
available <- list_survey_files()
download_survey_files(files = available$path[1])

## End(Not run)
```

extract_var *Extract the leading variable from a condition*

Description

Extract the leading variable from a condition

Usage

```
extract_var(x)
```

Arguments

x	One condition string.

Value

A character scalar naming the left-hand operand of a comparison, between, or membership expression; NA for unsupported or invalid syntax.

filter_second *Remove rows with a missing second-column value*

Description

Remove rows with a missing second-column value

Usage

```
filter_second(.)
```

Arguments

.	A data frame with at least two columns.

Value

The input data frame with rows containing NA in column two removed.

Examples

```
filter_second(data.frame(label = c("A", "B"), value = c(NA, 3)))
```

get_blank_cols *Find blank worksheet header columns*

Description

Find blank worksheet header columns

Usage

```
get_blank_cols(tab)
```

Arguments

tab	Wide data frame containing row_index and Excel columns named 1, 2, etc. Header rows 3 through 8 are inspected.

Value

Sorted integer column indices, excluding column 1. NA, empty text, and the string NA count as blank.

install_prep_dependencies *Install missing preparation-script dependencies*

Description

Install missing preparation-script dependencies

Usage

```
install_prep_dependencies(
  prep_script,
  repos = "https://cloud.r-project.org",
  lib = .libPaths()[[1L]]
)
```

Arguments

prep_script	Path to a preparation R script.
repos	Package repository URL, defaulting to the CRAN cloud mirror.
lib	Installation library. Defaults to the first active library. It must be listed in .libPaths().

Details

Maintained survey dependencies install with micsPlusTableR. This function is only needed for repair or extra custom-script packages.

`check_prep_dependencies()` identifies missing or unloadable packages. Only those packages are installed. Transitive required dependencies are installed by R. Preparation scripts are never executed by this function.

Installation happens only when this function is explicitly called.

`prepare_mics_data()` reports missing packages without installing them.

Value

The dependency check data frame, invisibly, after installation.

See Also

`check_prep_dependencies()`, prep-script-dependencies

---

list_survey_files *List survey tabulation plans and preparation files*

Description

Discover or download tabulation plans, preparation scripts, and their supporting reference assets from the separate survey files repository. Household and household-member microdata are not distributed here; obtain them separately from the MICS Plus website with the required permission. The repository must be public. No GitHub account or token is needed for these plans and preparation files.

Usage

```
list_survey_files(repo = "bariscr/micsPlusTableR-Files", ref = "main")
```

Arguments

`repo`	GitHub repository in `owner/name` form.
`ref`	Branch, tag, or commit to download. Defaults to `"main"`.

Details

Survey folders use the country, survey period, and wave, such as `Jamaica (2023-24) Wave 1`. Legacy code-and-wave folders are also supported. Downloads include root-level tabulation plans (`.xls`, `.xlsx`) and `.R` scripts or `.RData` reference assets inside `prep-files-*` subfolders. Other files, including SPSS microdata and hidden files, are excluded. Preparation subfolders are preserved. A single Git commit is resolved before listing or downloading, so changes to the branch during a download cannot mix remote versions. Newly published survey folders are discovered without a package update.

Downloads are staged and their sizes checked before destination files are written. A network failure leaves existing destination files unchanged. Files are downloaded in binary mode and are never executed. Selecting individual files may omit preparation helpers; select a whole survey folder for

all its published materials. Microdata must still be obtained separately. Git LFS pointers are not supported.

Internet access is required. Unauthenticated GitHub API rate limits apply; if a request fails, check the connection and public repository/ref, or retry later. The browser ZIP link in the user's guide is an alternative.

Value

`list_survey_files()` returns a data frame with `survey`, `path`, `size` (bytes), and `url` columns, plus a `commit` attribute.

See Also

`download_survey_files()`

Examples

```
## Not run:
available <- list_survey_files()
unique(available$survey)
available$path

## End(Not run)
```

---

mics_session *Create an isolated MICS Plus tabulation session*

Description

Holds prepared data, a loaded plan, engine functions, and results. The app creates its own session automatically. Use this function directly for code-only workflows, diagnosis, or tests; see offline-workflow.

Usage

```
mics_session(hh = NULL, hl = NULL)
```

Arguments

`hh`	Optional prepared household data frame.
`hl`	Optional prepared household-member data frame.

Details

A session is an ordinary R environment that keeps related objects together. Creating one does not start Shiny, open a browser, or launch the app. Pass the same session to subsequent workflow functions so they can use its loaded plan and data. Use separate sessions for independent tests.

An empty session is enough to read and inspect a plan with `read_mics_tabulation()`. Calculating results also requires the prepared datasets referenced by the plan. Supply already prepared `hh` and `hl` objects here, or add them later with `session$hh <- hh` and `session$hl <- hl`. These assignments do not run a preparation script; variables and weights used by the plan must already exist in those data. Use `prepare_mics_data()` when starting from raw survey files instead.

Value

A mutable, isolated tabulation session environment.

Examples

```
# Creates an R object; no app or browser is started.
session <- mics_session()
session
## Not run:
# If hh and hl are already prepared data frames in your workspace:
session <- mics_session(hh = hh, hl = hl)

## End(Not run)
```

---

mics_table_name *Get the current table title*

Description

Get the current table title

Usage

```
mics_table_name(session)
```

Arguments

session	A session with a tabulation sheet already read.

Value

The table title as a character scalar.

---

normalize_condition_text *Normalize condition text copied from Excel*

Description

Normalize condition text copied from Excel

Usage

```
normalize_condition_text(x)
```

Arguments

x	Character vector of conditions.

Value

Trimmed text with smart quotes, nonbreaking spaces, and Unicode comparison/logical operators replaced by their R equivalents; NA is retained.

`offline-workflow` *Run MICS Plus tabulation workflows outside Shiny*

Description

The application is the main user interface. These supporting functions expose its packaged engine for reference, tests, and workflows run directly in R. They are not additional steps required when using the app. A session is an ordinary R environment that keeps survey data, plan context, extra tables, and calculated results together. Creating it with `mics_session()` does not start Shiny, open a browser, or launch the app.

Details

Here, outside Shiny means running without the interface, not a guarantee that every operation is offline. Installing packages or downloading survey files requires internet. With local packages and inputs, table calculations and workbook output can run offline.

Workflow functions such as `read_mics_tabulation()` require an explicit `session` argument with no default. Create it once and pass the same object to subsequent calls. Use a separate session for each independent test. The examples require your own Excel plan, and prepared data or survey files when calculating results. Replace the example paths and sheet names with yours. See the package overview for links to the function references and export steps.

Read and inspect a plan without survey data

Create an empty session with `mics_session()`, then pass it to `read_mics_tabulation(session, path_tab_excel, sheet)`. Only the Excel plan is needed for this step. The returned list includes `filter_row` for filter instructions and `tab` for the cell statistics and conditions. The plan is also kept in the session for subsequent calculations.

Calculate with already prepared data

If `hh` and `hl` are already prepared data frames in your workspace, add them to the session holding the plan with `session$hh <- hh` and `session$hl <- hl`. Alternatively, start with `mics_session(hh = hh, hl = hl)` and then read the plan. Supply the datasets referenced by the plan; a household-only plan can use `hh` alone. These operations do not prepare or transform the data. The variables and weights required by the plan must already be present. Call `tabulate_mics_table(session)` to calculate results, then `pivot_mics_table(session, results, type = "header")` to review them.

Start from raw survey files

Create a session and call `prepare_mics_data()` with your household file, household-member file, and survey preparation script. Then read and calculate the plan, check consistency, and pivot the results before writing them.

Inspect individual steps

Stateful helpers take a session as their first argument. Read the plan and inspect its row and column predicates before calculating. Prepared data is needed for calculation, but not for reading the plan or parsing its predicates. `tabulate_mics_table()` selects the direction automatically and additionally attaches variable-explanation labels.

Use raw long-format numeric results for consistency checks, rather than pivoted or formatted tables. Each individual check returns `status`, `issue_count`, `details` (all comparisons), and `issues` (failing comparisons). A `not_applicable` result means the required total or distribution is absent; it is not a passing validation. `check_mics_table()` runs all six checks with a default tolerance of `1e-6`; individual helpers default to `1e-11`.

Calculation errors identify the worksheet, block or cell, and expression when available. A missing-variable message means an expression refers to a column absent from that block's prepared data. Correct the named input and rerun. Checks cannot validate the substantive choices in a preparation script or plan.

See Also

micsPlusTableR-package for the package overview.

`calc_cells` for a runnable weighted-count example.

statistic-types and statistic-precision for statistic reference.

`download_survey_files` for survey-material download examples.

Examples

```
## Not run:
library(micsPlusTableR)

# 1. Read and inspect a plan without survey data or the app.
session <- mics_session()
plan <- read_mics_tabulation(
  session = session,
  path_tab_excel = "TabulationPlan.xlsx",
  sheet = "6.2"
)
plan$filter_row
plan$tab

# 2. Continue with hh and hl already prepared in your workspace.
session$hh <- hh
session$hl <- hl
results <- tabulate_mics_table(session)
preview <- pivot_mics_table(session, results, type = "header")

# Alternatively, supply prepared data when creating a new session.
prepared_session <- mics_session(hh = hh, hl = hl)
results <- tabulate_mics_table(prepared_session, "TabulationPlan.xlsx", "6.2")

# 3. Start from raw files instead, using your survey preparation script.
session <- mics_session()
prepare_mics_data(
  session,
  "hh.sav",
  "hl.sav",
  prep_script = "survey-preparation/survey_prep.R",
  output_dir = "micsPlusTableR-output"
)
result <- tabulate_mics_table(session, "TabulationPlan.xlsx", "1.1")
checks <- check_mics_table(session, result)
preview <- pivot_mics_table(session, result, type = "header")
# Export only when needed; this function requires an existing destination.
```

```
workbooks <- create_excel_workbooks(
  "TabulationPlan.xlsx", "micsPlusTableR-output",
  country_code = "JAM", period = "2023-24", wave = "Wave 1"
)
write_mics_table(session, workbooks$output, sheet = "1.1", table = result)
write_mics_footnotes(session, workbooks$output, sheet = "1.1", table = result)

# Alternatively, inspect and run individual steps after preparing the session.
plan <- read_mics_tabulation(session, "TabulationPlan.xlsx", "1.1")
row_rules <- row_condition_f(plan$tab_r)
column_rules <- col_condition_f(plan$tab_c)
result <- if (plan$tab_direction == "v") tabulate_v(session) else tabulate_h(session)
row_check <- row_group_total_check(session, result, diff = 1e-6)
row_check$status
row_check$issues

## End(Not run)
```

`package-dependencies` *Package runtime and development dependencies*

Description

Required Imports cover the app, tabulation engine, and maintained preparation scripts. They install together. Preparation-only packages are documented separately in prep-script-dependencies.

Details

Runtime packages and their purposes:

- `shiny`, `bslib`, `shinyFiles`: application UI, server, and file selection.
- `htmltools`, `htmlwidgets`, `reactable`: HTML controls and table displays.
- `flextable`, `officer`: formatted table previews and borders.
- `openxlsx2`: Excel output workbooks and formatting.
- `readxl`, `tidyxl`: worksheet values, cell formats, and plan parsing.
- `dplyr`, `tidyr`, `tibble`: table transformation and cell results.
- `purrr`, `rlang`, `stringr`: iteration, expressions, and condition parsing.
- `haven`: display and clean labelled survey values in the app.
- `janitor`: remove empty columns from comparison workbooks.
- `here`: locate the user's project and default output folder.
- `jsonlite`: read the remote survey-file listing.
- `readr`: CSV downloads of long-format results.
- `later`: schedule application UI updates.

The development-only Suggests entry is testthat for tests. Developer tools such as roxygen2, pkgload, and remotes are installed separately as described in the README. Rebuilding the User's Guide requires Quarto and XeLaTeX; its rendered HTML and PDF are bundled for users.

The maintainer command `Rscript tools/check-dependencies.R` inspects executable app and engine code, qualified references, and imported function calls. It rejects undeclared direct references and Imports with no detected usage in the app, engine, or preparation namespace loaders. It also checks that all documented preparation dependencies are required Imports. Both package and reference-manual builds run this audit.

See Also

prep-script-dependencies, micsPlusTableR-package

`pivot_mics_table` *Pivot long-format cell results for review*

Description

Pivot long-format cell results for review

Usage

```
pivot_mics_table(
  session,
  table = NULL,
  formatted = FALSE,
  type = c("index", "header", "logic")
)
```

Arguments

`session`	A tabulation session.
`table`	Cell results; defaults to the session's latest result.
`formatted`	`FALSE` uses raw numeric `value`; `TRUE` uses the engine's `value_f` display strings; `"view"` uses preview strings in `value_f_view`.
`type`	`"index"` uses worksheet row/column positions, `"header"` uses descriptive labels, and `"logic"` uses row/column conditions. The choice changes labels, not calculated values.

Value

A wide review table.

`prep-script-dependencies`
Preparation script dependencies

Description

Survey preparation packages are tracked separately from the packages needed to launch the app and run the tabulation engine. Both groups are required Imports and install together with micsPlusTableR; no separate setup is needed.

Details

The maintained preparation scripts declare `prep_dependencies` near the top of the file.

`prepare_mics_data()` checks these packages and the detected helper dependencies before executing the script. The Shiny Data Preparation workflow uses the same check. Missing or unloadable packages produce an error naming the package, the script requiring it, and an installation command.

Use `check_prep_dependencies()` for diagnosis.

Use `install_prep_dependencies()` for repair or extra custom-script packages. Neither is a required step for normal installation or switching between maintained surveys.

Additional DESCRIPTION field: `Config/micsPlusTableR/prep-packages`.

Preparation-only packages in the maintained Jamaica workflows:

- `ggplot2`: FIES distribution and thermometer plots.
- `Hmisc`: weighted prevalence means in the FIES helpers.
- `memisc`: SPSS import retaining value labels for the FIES routines.
- `reshape2`: reshape FIES plotting data with melt.
- `RM.weights`: weighted Rasch model and food-insecurity calculations.
- `survey`: survey designs and design-based FIES estimates.
- `srvyr`: survey-design pipelines in the main preparation scripts.
- `labelled`: variable/value labels in Jamaica Wave 2 recodes.

All eight packages are installed by the normal package installation command. They are not imported into the app's namespace. Packages required by the selected script are made available in its private preparation environment; package functions already supplied by the core workflow retain precedence. Use explicit package::function calls if a script needs a conflicting name.

Shared dependencies are also declared by the scripts: Mongolia Waves 1 and 2 use dplyr and haven. Jamaica Waves 1 and 2 also use here, tibble, and tidyr; Wave 2 additionally uses purrr. These are already package Imports.

Keep declarations current when editing scripts or adding helpers. The checker scans the main script, all R files in its sibling FIES-inputs directory, and literal source paths. Declare packages used through dynamically constructed names or source paths explicitly. The checker does not execute preparation code and does not certify statistical results or package-version compatibility.

See Also

`prepare_mics_data()`, package-dependencies

Examples

```
## Not run:
prep_script <- "survey-preparation/survey_prep.R"
check_prep_dependencies(prep_script)
# For repair or additional dependencies in a custom script only:
install_prep_dependencies(prep_script)

## End(Not run)
```

prepare_mics_data *Run a survey preparation script outside the Shiny application*

Description

Run a survey preparation script outside the Shiny application

Usage

```
prepare_mics_data(session, hh_path, hl_path, prep_script, output_dir = NULL)
```

Arguments

session	A session created by mics_session().
hh_path	Path to the household .sav file.
hl_path	Path to the household-member .sav file.
prep_script	Path to the trusted preparation .R script. A sibling FIES-inputs folder is detected automatically.
output_dir	Main output folder. If the script's folder contains FIES-inputs, FIES artifacts are written to FIES-outputs below this folder.

Details

Checks declared and detected preparation dependencies before running any preparation code. Maintained survey dependencies install with the package. install_prep_dependencies() is available for repair or extra custom-script requirements. See prep-script-dependencies for the separate dependency list.

Value

The session, invisibly.

read_mics_tabulation *Read one sheet from a MICS Plus tabulation plan*

Description

Reads the source name in B4:B7, filters, weights, conditions, cell statistics, headers, and layout metadata into the session. Blank statistic cells are filled by the worksheet reader. Direction is horizontal when an n_unw instruction immediately precedes IDX or a 100/100.0 statistic is present; otherwise it is vertical. Optional d= does not change that decision. The reader converts recognized numbered total column aliases to total == 1 and totalHL column aliases to hhmembers. See worksheet-conditions, statistic-types, statistic-precision, and tabulate_h().

Usage

```
read_mics_tabulation(session, path_tab_excel, sheet)
```

Arguments

session	Required session created by mics_session().
path_tab_excel	Path to the Excel tabulation plan.
sheet	Sheet name or index.

Details

This function runs directly in R without the app. The session argument is required and has no default. Create it with mics_session(), which creates an R environment without starting Shiny or opening a browser. Reading and inspecting a plan does not require survey data. To calculate results afterward, add the prepared datasets referenced by the plan to the same session, then call tabulate_mics_table(). See offline-workflow for examples using existing prepared data or raw survey files.

Value

The parsed tabulation-plan context.

Examples

```
## Not run:
# Only the Excel plan is needed for this inspection.
session <- mics_session()
plan <- read_mics_tabulation(
  session = session,
  path_tab_excel = "TabulationPlan.xlsx",
  sheet = "6.2"
)
plan$filter_row
plan$tab

# To calculate, add your already prepared data to the same session.
session$hh <- hh
session$hl <- hl
results <- tabulate_mics_table(session)
pivot_mics_table(session, results, type = "header")

## End(Not run)
```

---

read_previous_mics_table

Read a previously generated worksheet for comparison

Description

Read a previously generated worksheet for comparison

Usage

```
read_previous_mics_table(path, sheet, skip = 3)
```

Arguments

path	Excel workbook path.
sheet	Sheet name or index.
skip	Number of header rows to skip.

Value

A cleaned table.

---

row_condition_f *Parse row conditions from a plan*

Description

Parse row conditions from a plan

Usage

```
row_condition_f(tab_r)
```

Arguments

tab_r	Data frame containing row_index and row_lgc.

Details

Separates mutate calls from predicates; blank or NA row logic means TRUE. Supports nested and multiline calls, multiple mutations, and - - - between setup and the complete multiline predicate. Legacy inline pipe-separated row instructions are also accepted. Parses text without evaluating it against survey data.

Value

A tibble containing the row index, original condition, cleaned condition, generated variable name, and calculation.

See Also

col_condition_f(), normalize_condition_text(), worksheet-conditions

row_group_perc_total_check *Check that row-group percentages sum to 100*

Description

Check that row-group percentages sum to 100

Usage

```
row_group_perc_total_check(session, cell_results = NULL, diff = 1e-11)
```

Arguments

session	A tabulation session. Indent checks also require the loaded plan's indent_rows metadata.
cell_results	Long-format numeric results, defaulting to the latest result in the session. Use the complete results with logic/group metadata.
diff	Maximum absolute difference accepted as consistent.

Details

Runs only this check and updates its corresponding session check state. A missing total or distribution produces not_applicable. Use check_mics_table() to run all six checks together.

Value

A list with status (pass, fail, or not_applicable), issue_count, details (all comparisons), and issues (failing comparisons).

row_group_total_check *Compare row-group counts with the overall total*

Description

Compare row-group counts with the overall total

Usage

```
row_group_total_check(session, cell_results = NULL, diff = 1e-11)
```

Arguments

session	A tabulation session. Indent checks also require the loaded plan's indent_rows metadata.
cell_results	Long-format numeric results, defaulting to the latest result in the session. Use the complete results with logic/group metadata.
diff	Maximum absolute difference accepted as consistent.

Details

Runs only this check and updates its corresponding session check state. A missing total or distribution produces not_applicable. Use `check_mics_table()` to run all six checks together.

Value

A list with `status` (pass, fail, or not_applicable), `issue_count`, `details` (all comparisons), and `issues` (failing comparisons).

`row_indent_group_perc_total_check`

Compare indented child percentages with their parent row

Description

Compare indented child percentages with their parent row

Usage

```
row_indent_group_perc_total_check(session, cell_results = NULL, diff = 1e-11)
```

Arguments

`session`	A tabulation session. Indent checks also require the loaded plan's `indent_rows` metadata.
`cell_results`	Long-format numeric results, defaulting to the latest result in the session. Use the complete results with logic/group metadata.
`diff`	Maximum absolute difference accepted as consistent.

Details

Runs only this check and updates its corresponding session check state. A missing total or distribution produces not_applicable. Use `check_mics_table()` to run all six checks together.

Value

A list with `status` (pass, fail, or not_applicable), `issue_count`, `details` (all comparisons), and `issues` (failing comparisons).

row_indent_group_total_check

Compare indented child counts with their parent row

Description

Compare indented child counts with their parent row

Usage

```
row_indent_group_total_check(session, cell_results = NULL, diff = 1e-11)
```

Arguments

session	A tabulation session. Indent checks also require the loaded plan's indent_rows metadata.
cell_results	Long-format numeric results, defaulting to the latest result in the session. Use the complete results with logic/group metadata.
diff	Maximum absolute difference accepted as consistent.

Details

Runs only this check and updates its corresponding session check state. A missing total or distribution produces not_applicable. Use check_mics_table() to run all six checks together.

Value

A list with status (pass, fail, or not_applicable), issue_count, details (all comparisons), and issues (failing comparisons).

run_app

Run the micsPlusTableR Shiny application

Description

The main user entry point to micsPlusTableR. Opens the browser-based MICS Plus Tabulation app for preparing inputs, calculating and checking tables, exploring data, and saving results. Other package functions mainly expose its supporting engine for documentation, tests, and code-only workflows.

Usage

```
run_app(
  host = "127.0.0.1",
  port = getOption("shiny.port"),
  launch.browser = interactive(),
  output_dir = NULL,
  ...
)
```

Arguments

host	Host interface passed to shiny::runApp().
port	Port passed to shiny::runApp(); NULL selects a free port.
launch.browser	Whether to open the application in a browser.
output_dir	Default directory for generated Excel workbooks. When NULL, the app creates micsPlusTableR-output in the project root detected by here::here().
...	Additional arguments passed to shiny::runApp().

Details

The app opens on Data Preparation. User's Guide provides embedded instructions, Open full guide, and Download PDF. Download Files saves plans and preparation materials to the current project by default.

Results can be inspected without exporting. Single-table Excel writing needs a created or selected workbook. Multi-Sheet Tabulator automatically creates required workbooks for unset destinations and reuses destinations already set in Write to Excel. The default output folder is below the survey project; output_dir selects another location. Existing destinations and uploaded inputs must be selected again after restarting the app.

Restart the R session after installing or updating the package before launching, so previously loaded functions are not retained in memory. The external SheetJS browser library is needed for the workbook preview; local calculations and Excel output do not depend on that download.

Value

Invisibly returns the value from shiny::runApp().

See Also

micsPlusTableR-package, offline-workflow

---

statistic-precision *Worksheet display precision*

Description

Every supported worksheet statistic accepts an optional named d argument specifying the number of decimal places to display.

Details

Use mean(HCS8, d=0) for a whole-number expense mean, mean(d=1) for a mean indicator with one decimal place, p(d=2) for two percentage decimals, or n(d=1) for a count with one displayed decimal. Existing argument forms retain their arguments: median(age, d=2), mean_unw(age, d=0), p_sum(x, d=2), and p(100, d=2). A constant may use 100(d=2). The statistic must be supported in the table's direction; adding d does not change the calculation or its weighting.

d must be a literal non-negative whole number, supplied once. Zero means no decimal point. Omit d to keep the existing formatting for that statistic and output mode; means still default to one decimal place. Spaces around arguments are allowed. d means decimal places, not significant figures.

The Formatted and Suppressed Tabulator views honor explicit precision for visible numbers; Unformatted retains full numeric estimates. Excel applies number formats in both output workbooks, including multi-sheet runs. Parentheses retain the requested precision; `(*)` and `-` remain markers. Display rounding does not alter stored estimates, calculations, weights, denominators, suppression thresholds, or consistency checks.

Internally, `stat_type` contains the statistic without `d`; `display_digits` holds the explicit integer or NA when omitted. Worksheet filling carries the statistic and its precision together. Changing only `d` does not change mean filter boundaries or the identification of `n_unw` before IDX. Long-format output uses the override in `DECIMALS` and display strings while retaining full precision in `OBS_VALUE`.

See Also

`tabulate_h()`, `tabulate_v()`, `write_mics_table()`

`statistic-types` *Worksheet statistic types and populations*

Description

A worksheet's `stat_type` specifies the calculation for a result cell. These instructions drive the application and the R tabulation engine. The User's Guide's Understand the tabulation plan reference introduces the worksheet layout, dataset, filters, conditions, and worked examples.

Cell scope and direction

Statistics are written below the condition row, from Excel column C onward. The worksheet reader fills blank statistic cells from preceding instructions; a blank is not necessarily a skipped cell or a zero. Direct `calc_cells()` calls instead return NA for missing statistic specifications.

Dataset and active filters first define the available records. In horizontal tables, the row condition selects the percentage base and the column condition defines its outcome. In vertical tables, the column condition selects the base and the row condition defines the outcome. Counts, numeric means, and medians use records satisfying both conditions. See worksheet-conditions for preparing predicates, numbered total aliases, and household-member bases.

A percentage denominator comes from the applicable population, not a nearby count cell. Weighted percentages are 100 times the sum of outcome weights divided by the sum of base weights. Normal application tabulation uses the plan's weight; the low-level `calc_cells()` helper also has an explicit `weighted` argument for tests and code-only calculations.

Counts and indicator percentages

`n` Weighted count: sum of weights for qualifying records. Both directions.

`n_unw` Unweighted count of qualifying records. Both directions. In horizontal tables, the special `hhmembers` column condition instead sums `HLnum` for qualifying household records with `total == 1`. The worksheet column alias `totalHL == 1` selects this behavior; it does not change the weight or apply to vertical tables.

`p` Weighted percentage of the outcome within the base. Both directions.

`p_unw` Unweighted percentage of the row outcome within the column base. Vertical tables only.

`mean` Bare mean: the same weighted indicator-percentage calculation as `p`, on a 0-100 scale. It does not select a numeric variable. Both directions.

`mean_unw` Unweighted indicator percentage, on a 0-100 scale. Both directions.

Numeric summaries

`mean(variable)` Weighted arithmetic mean of the named numeric variable among records satisfying both conditions. Both directions.

`mean_unw(variable)` Unweighted arithmetic mean of that variable. Both directions.

`median(variable)` Unweighted median, even when the plan specifies a weight. Both directions. `median_unw(variable)` is a vertical-only synonym.

`mean(newvar = expression)` Vertical tables only: create the named variable from the expression, then calculate its weighted mean for the applicable records. For example, `mean(expense = HCS8 / 1000)`.

Missing values of the summarized variable are omitted. Numeric codes for unknown or inapplicable responses are not automatically missing.

Totals and specialized forms

`100`, `100.0` Constant 100, not an estimated percentage. Presence in a worksheet selects horizontal mode and can mark a distribution total. Zero-base and suppression rules can change the displayed value.

`p(100)`, `p_unw(100)` Vertical tables: return 100 on the first calculated row, then calculate weighted or unweighted percentages on later rows.

`n_unw(100)` Vertical tables: return 100 on the first calculated row, then unweighted counts on later rows. Later counts are not percentages.

`n(100)` Recognized only on the first calculated vertical row, returning 100. Use `n` on later rows. Repeating `n(100)` is unsupported.

`p_sum(variable)` Horizontal tables only: for records satisfying both conditions, calculate 100 times sum(variable) divided by sum(weight). The numerator is not multiplied by the weight; the prepared variable must be defined appropriately for this calculation.

`n1`, `n_unw1`, `p1` Calculation variants of `n`, `n_unw`, and `p` in both directions. Suffixes do not set decimal places.

The `(100)` forms are not supported by the horizontal calculator. They refer to the first calculated row, not any row whose label says Total. Prefer the ordinary count and percentage forms unless an existing plan needs a variant: identical calculations do not guarantee identical direction selection, suppression, count linkage, or consistency-check handling.

Missing conditions

Missing outcome conditions do not contribute to weighted percentage numerators, but their available weights remain in the base denominator. Horizontal unweighted indicator percentages treat missing outcomes as false; vertical unweighted indicator percentages omit missing indicators from their denominator. Define exclusions explicitly if unknown responses should be removed from the base.

Filters and display

Column B's filter remains global. Additional horizontal filters continue across every statistic type, including means. A new `filter(...)` replaces the secondary filter; `unfilter()` in the condition row clears it and restores the global-only population from that column onward. See `tabulate_h()`.

Every supported statistic accepts optional `d=` display precision without changing its calculation or direction support. For example, `mean(d=1)`, `mean(HCS8, d=0)`, and `p(100, d=2)`. See statistic-precision.

See Also

tabulate_mics_table(), statistic-precision

---

survey_choices *Maintain survey and wave choices*

Description

The plain UTF-8 CSV file 'inst/extdata/survey_choices.csv' in the package source controls the application's survey and wave selectors. Edit this source file directly before committing to GitHub or building the package.

Columns

Each row represents one available country, period, and wave combination. All four columns are read as character strings:

country Full country name, for example Jamaica.

country_code Exactly three uppercase letters, for example JAM.

period Survey period as text, for example 2023-24.

wave Wave label, for example Wave 1.

Keep the column names. The file must have at least one row, with no empty fields or duplicate country/period/wave combinations. All waves of the same country and period must use the same country code. Leading and trailing whitespace is removed. Row order determines selector order and defaults.

Editing and installation

Use a text editor or a spreadsheet editor that preserves the period and wave columns as text. Save as UTF-8 CSV. From the package root, run source("run_dev.R") to load and launch the development source. Stop and restart the app after saving changes.

The CSV requires no conversion to an R data object.

R installs 'inst/extdata' files into the package's 'extdata' directory. For installed packages, reinstall the updated source and restart the R session before launching the app. Direct edits to an installed copy are replaced by reinstallation or upgrades. The selected code, full period, and wave also contribute to output filenames and long-format metadata.

See Also

run_app, micsPlusTableR-package

Examples

```
path <- system.file("extdata", "survey_choices.csv", package = "micsPlusTableR")
choices <- read.csv(path, colClasses = "character", fileEncoding = "UTF-8-BOM")
choices
```

tabulate_extra_table *Map a supplied extra table to the current plan*

Description

Map a supplied extra table to the current plan

Usage

```
tabulate_extra_table(session, table)
```

Arguments

session	A prepared session with a loaded plan.
table	A data frame whose first column contains row labels and whose remaining columns contain values in plan order.

Details

Row logic is retained as metadata, not evaluated against survey data. Values are mapped by position and the engine applies suppression. The plan's optional d= argument controls display precision for supplied values too; see statistic-precision.

Value

Long-format results, also saved in session$cell_results.

tabulate_h *Calculate a horizontal table*

Description

Process column filter blocks and calculate column indicators using the currently loaded horizontal plan. The direction must match the plan. Results are also saved in session$cell_results. Use tabulate_mics_table() for automatic direction selection and additional variable-explanation labels.

Usage

```
tabulate_h(session, skip_row_conditions = FALSE)
```

Arguments

session	A session created by mics_session() with prepared data and a plan loaded by read_mics_tabulation().
skip_row_conditions	Logical; replace row predicates with TRUE during calculation and skip row mutations while retaining their labels. Intended for extra-table testing.

Details

The condition-row `filter(...)` in Excel column B applies globally. Each additional horizontal `filter(...)` starts at its own column and combines with B. It extends right across all statistic types, including means, until the next explicit `filter(...)` or `unfilter()` instruction in the condition row. A new filter replaces the previous secondary filter. Write `unfilter()` in place of `filter(...)` to clear the secondary filter from that column onward and use only B's global filter until a new filter starts. Row and column predicates still apply. Blank filter cells, changes of statistic or `d=`, and changes down rows do not stop a filter. Filters run before calculations. Within each filter block, condition-row `mutate(...)` instructions run in column order. Each mutation is available to its own column and following columns in that block; a later redefinition never changes earlier cells. Column B's calculation runs once at the start of each block. Mutations do not start or stop a secondary filter or change its count-linking boundaries. They use temporary data and do not modify the session's prepared `hh` or `hl`. Vertical tabulation keeps its separate primary-filter behavior; see `tabulate_v()`. Row mutations run top to bottom independently within each column calculation stage, before row predicates. See statistic-types for calculation rules. Display precision `d=` is independent of filter scope; see statistic-precision.

Value

A long-format cell-results data frame.

See Also

`tabulate_v()`, `tabulate_mics_table()`

`tabulate_mics_table` *Tabulate a MICS Plus table outside the application*

Description

Exposes the app's calculation engine for code-only workflows and tests. Selects the loaded plan's direction automatically or maps an available supplied extra table, saves results in the session, and attaches variable explanations. Use numeric long-format results for checks before pivoting. See statistic-types for calculations and offline-workflow for an example.

Usage

```
tabulate_mics_table(session, path_tab_excel = NULL, sheet = NULL)
```

Arguments

`session`	A prepared tabulation session.
`path_tab_excel`	Optional tabulation-plan path. Supply with `sheet` to read a new table before tabulating.
`sheet`	Optional sheet name or index.

Value

A long-format data frame containing one row per calculated cell.

tabulate_v *Calculate a vertical table*

Description

Filter prepared data by column and calculate row indicators using the currently loaded vertical plan. The direction must match the plan. Results are also saved in `session$cell_results`. Use `tabulate_mics_table()` for automatic direction selection and additional variable-explanation labels.

Usage

```
tabulate_v(session, skip_row_conditions = FALSE)
```

Arguments

`session`	A session created by `mics_session()` with prepared data and a plan loaded by `read_mics_tabulation()`.
`skip_row_conditions`	Logical; replace row predicates with TRUE during calculation and skip row mutations while retaining their labels. Intended for extra-table testing.

Details

Shared column-B calculations run after the primary filter. Row mutations then run top to bottom, before column and row predicates. Each mutation is available to its own and subsequent rows; later redefinitions do not affect earlier results. See worksheet-conditions for setup syntax. Every supported statistic accepts an optional `d=` display argument; for example, `mean(x, d=0)`, `mean(d=1)`, or `p(d=2)`. Omitting it preserves existing formatting. See statistic-types for each calculation, population, weighting rule, and direction limit, and statistic-precision for display rules.

Value

A long-format cell-results data frame.

See Also

`tabulate_h()`, `tabulate_mics_table()`

validate_sdmx_compatible *Validate SDMX-compatible MICS long-format data*

Description

Checks the complete 35-variable contract and the unique observation key: `SURVEY_ID`, `TABLE_ID`, `row_order`, `column_order`, and `TIME_PERIOD`.

Usage

```
validate_sdmx_compatible(data)
```

Arguments

data	A data frame produced by as_sdmx_compatible().

Value

data, invisibly.

---

worksheet-conditions *Worksheet row and column conditions*

Description

Prepare conditions around the required output's unit, eligible population, comparison groups, and statistic. The User's Guide section Define row and column conditions for your output provides worked worksheet layouts.

Location and population

Row predicates are written in column B below the condition row. Column predicates are written from C onward on the condition row identified by the source instruction in B4:B7. Labels alone do not select records. Ordinary predicates use prepared variables, for example `area == 1`, `age >= 15 & age <= 49`, or `response %in% c(1, 2)`.

After dataset selection and filters, horizontal percentages use the row predicate as the base and the column predicate as the outcome. Vertical percentages use the column predicate as the base and the row predicate as the outcome. Counts and numeric summaries normally use both predicates. A count in a neighboring cell does not define a percentage's denominator. Missing-outcome handling is documented in statistic-types.

The worksheet reader omits blank row-condition cells. The standalone row_condition_f() helper instead treats blank predicates as TRUE. Column instructions can fill from the preceding column. Write changed predicates explicitly. In a multiline column instruction, use a separator `- - -` after source/filter/weight/calculation instructions. The complete predicate below the separator can span multiple lines.

Creating variables in the condition row

Use `mutate(...)` above `- - -` to create variables for column conditions. For example:

```
mutate(WS1R = if_else(WS1 %in% c(11, 12), 1, 0))
- - -
WS1R == 1 &
between(WS4, 1, 30)
```

Within a horizontal filter block, a mutation applies at its own column and continues right. A later mutation can use an earlier variable or redefine it for subsequent columns without changing earlier results. Column B's calculation is shared by the filter blocks and runs once per block. A mutation does not change which secondary filter is active. Filters still run before these calculations. Use the preparation script for variables needed by filters or across different filter blocks. Worksheet mutations operate on temporary data; they do not add columns to prepared `hh` or `hl`. Multiline and multiple `mutate(...)` instructions are supported.

The same syntax works in row-condition cells in column B, in both table directions. Row mutations run top to bottom before that row's predicate, and remain available to following rows. Later redefinitions do not affect earlier rows, and repeated mutations execute at each occurrence. A cell containing only mutations has predicate TRUE. Setup also runs on rows without a statistic. Horizontal row mutations run independently on each column calculation stage after its filters and column setup; vertical row mutations run after the primary filter and shared B setup, before column predicates. Row predicates do not restrict the data passed to later rows.

Total aliases in worksheet columns

Maintained preparation scripts set `total = 1` in household and member data. `total == 1` therefore selects the records remaining under the applicable filters and the other axis's predicate. The worksheet reader converts these exact column forms:

- `total1 == 1` through `total5 == 1` become `total == 1`.
- `totalHL == 1` and `totalHL1 == 1` through `totalHL4 == 1` become `hhmembers`.
- The legacy column form `all` becomes `total == 1`.

Numbered aliases accommodate repeated total/base columns in separate blocks. Their suffix does not select a subgroup, change weights, or set decimals; filters and source instructions define the block population. These names need not exist as prepared variables when used as recognized column aliases. Use the exact spelling and internal spacing shown above. Bare `total1`, additional suffixes, and compound expressions such as `total1 == 1 & area == 1` are not alias forms.

Conversion belongs to `read_mics_tabulation()`, not the predicate parsers or calculators. Aliases in row predicates or inside filters are not converted. Direct `calc_cells()` calls must use `total == 1` or the supported `hhmembers` condition explicitly.

Household-member bases

For a horizontal `n_unw` or `n_unw1` cell with column condition `hhmembers`, the calculator applies the row predicate and sums `if_else(total == 1, 1, 0, missing = 0) * HLnum`, with missing contributions removed. This counts people represented by household records. It requires prepared `total` and numeric, complete household-size `HLnum`. Two eligible household records with sizes 2 and 4 contribute 6 members; ordinary `total == 1` with `n_unw` contributes 2 household records.

Weighted member counts use ordinary `total == 1` with `n` and a suitable member-based weight, such as the approved household weight multiplied by `HLnum`. Member-based percentages use that weight with the desired outcome. `totalHL` and `hhmembers` do not create or select a weight. For sizes 2 and 4 and household weights 1 and 3, the weighted member count is 14, compared with 4 weighted households and 6 unweighted members.

The member-size special case does not apply to vertical calculations or other statistics. On one-record-per-member data, count qualifying records with ordinary predicates and `n_unw`; summing household sizes on every member record would overcount. Use member records for member-level subsets, such as children selected by age, rather than the full household-size sum.

Preparing repeated blocks

For a horizontal percentage reporting an outcome, use its predicate (for example `HCS7 == 1`) with `p`, followed by `total == 1` columns with `n` and `n_unw` for the base. To average a numeric amount only among records reporting it, place the eligibility/valid-value filter on the mean column, then `- - -` and `total1 == 1`. Use `mean(HCS8)` there and `n`/`n_unw` in following `total1 == 1` columns. The filter, not the numbered alias, creates the restricted population. See `tabulate_h()` for filter scope. Select survey-specific variables, valid-value rules, and weights before adapting this pattern.

A mean among positive reporters differs from a mean across all eligible records with correctly prepared zeros for non-reporters.

See Also

`read_mics_tabulation()`, statistic-types, `calc_cells()`

`write_mics_footnotes` *Add standard footnotes to a tabulation workbook*

Description

Add standard footnotes to a tabulation workbook

Usage

```
write_mics_footnotes(session, destination, sheet = NULL, table = NULL)
```

Arguments

`session`	A tabulation session.
`destination`	Existing destination workbook.
`sheet`	Sheet name or index; defaults to the current sheet.
`table`	Cell results; defaults to the latest result.

Value

The destination path, invisibly.

`write_mics_table` *Write cell results to a tabulation workbook*

Description

Write cell results to a tabulation workbook

Usage

```
write_mics_table(
  session,
  destination,
  sheet = NULL,
  table = NULL,
  formatted = FALSE,
  drop_n_unw = FALSE
)
```

Arguments

session	A tabulation session with a loaded plan.
destination	Existing destination workbook. Create it with create_excel_workbooks() or supply a previously created workbook.
sheet	Sheet name or index; defaults to the current sheet.
table	Cell results; defaults to the latest result.
formatted	Whether to write formatted display values.
drop_n_unw	Whether to omit unweighted-count columns when formatting.

Details

Writes to an existing workbook; this function does not create one. Automatic creation for unset destinations is part of the app's Multi-Sheet Tabulator workflow. Rewriting a sheet replaces its previously written values. Optional worksheet d= arguments set Excel decimal places in both output formats. Numeric values retain their precision; suppression markers and parentheses retain their meaning. See statistic-precision.

Value

The destination path, invisibly.

Index

∗ **documentation**
survey_choices, 32
∗ **package**
micsPlusTableR-package, 2

as_sdmx_compatible, 5
as_sdmx_compatible(), *4*, *36*

calc_cells, 5, *19*
calc_cells(), *3, 4*, *30*, *37, 38*
check_mics_table, 6
check_mics_table(), *3*, *9*, *26–28*
check_prep_dependencies, 7
check_prep_dependencies(), *4*, *15*, *22*
col_condition_f, 8
col_condition_f(), *3*, *25*
col_group_perc_total_check, 8
col_group_perc_total_check(), *4*
col_group_total_check, 9
col_group_total_check(), *4*
compare_mics_tables, 10
compare_mics_tables(), *3*
create_excel_workbooks, 10
create_excel_workbooks(), *3*, *39*

download_survey_files, 11, *19*
download_survey_files(), *16*

extract_var, 13
extract_var(), *3*

filter_second, 13
filter_second(), *3*

get_blank_cols, 14
get_blank_cols(), *3*

install_prep_dependencies, 14
install_prep_dependencies(), *4*, *7*, *22, 23*

list_survey_files, 15
list_survey_files(), *11, 12*

mics_session, 16
mics_session(), *3*, *23, 24*, *33*, *35*
mics_table_name, 17
micsPlusTableR *(*micsPlusTableR-package*)*, 2
micsPlusTableR-package, 2, *19*, *21*, *29*

normalize_condition_text, 17
normalize_condition_text(), *3*, *8*, *25*

offline-workflow, *3, 4*, *16*, 18, *24*, *29*, *34*

package-dependencies, *4*, 20, *22*
pivot_mics_table, 21
pivot_mics_table(), *3*
prep-script-dependencies, *4*, *7*, *15*, *20*, 21, *21*, *23*
prepare_mics_data, 23
prepare_mics_data(), *3*, *7*, *15, 16*, *22*

read_mics_tabulation, 23
read_mics_tabulation(), *3*, *6*, *8*, *16*, *33*, *35*, *37, 38*
read_previous_mics_table, 24
read_previous_mics_table(), *3*
row_condition_f, 25
row_condition_f(), *3*, *8*, *36*
row_group_perc_total_check, 26
row_group_perc_total_check(), *3*
row_group_total_check, 26
row_group_total_check(), *3*
row_indent_group_perc_total_check, 27
row_indent_group_perc_total_check(), *4*
row_indent_group_total_check, 28
row_indent_group_total_check(), *4*
run_app, 28, *32*
run_app(), *2–4*

statistic-precision, *4*, *6*, *19*, *23*, 29, *31–35*, *39*
statistic-types, *4*, *6*, *19*, *23*, 30, *34–36*, *38*
survey_choices, *4*, 32

tabulate_extra_table, 33
tabulate_extra_table(), *3*
tabulate_h, 33
tabulate_h(), *3*, *23*, *30, 31*, *35*, *37*

tabulate_mics_table, 34
tabulate_mics_table(), *3*, *24*, *32–35*
tabulate_v, 35
tabulate_v(), *3*, *30*, *34*

validate_sdmx_compatible, 35
validate_sdmx_compatible(), *4*

worksheet-conditions, *6*, *8*, *23*, *25*, *30*, *35*, 36
write_mics_footnotes, 38
write_mics_footnotes(), *3*
write_mics_table, *11*, 38
write_mics_table(), *3*, *30*